DuocUC | ONLINE



Desarrollo Backend III

Informe técnico: migración del Banco XYZ a microservicios con Spring Cloud y Spring Batch

Evaluación Final Transversal · Diego Alvarez · 11 de octubre de 2026

Índice

1. Resumen ejecutivo....3

2. Contexto, procesos clave y requerimientos....3

3. Propuesta de arquitectura....4

4. Parte 1: procesos batch con Spring Batch....6

5. Parte 2: Backend for Frontend....7

6. Parte 3: microservicios con Spring Cloud....8

    6.1 Config, Eureka, balanceo y seguridad....8

    6.2 Resilience4j y comportamientos alternativos....8

    6.3 Kafka y consistencia de datos....9

7. Docker, escalado horizontal y despliegue en AWS....11

    7.1 Verificación de Kafka en EC2....13

8. Resultados y comparación con el sistema legacy....14

9. Desafíos y soluciones....15

10. Mejoras y próximos pasos....15

11. Anexos....16

    A. Repositorio y documentación....16

    B. Diagramas complementarios....16

    C. Capturas....21

1. Resumen ejecutivo

El Banco XYZ opera sobre una plataforma legacy en COBOL y scripts Shell en mainframe. Este proyecto propone e implementa su migración a microservicios con Spring Boot 3.5 y Spring Cloud 2025.0. Cubre las tres partes del encargo: los tres procesos batch reescritos en Spring Batch, un BFF por canal (web, móvil y cajero) y los microservicios de cuentas, pagos y clientes, con Eureka, Spring Cloud Config, OAuth2, Resilience4j y eventos en Kafka.

Todo corre con un docker-compose.yaml y entra por un api-gateway en HTTPS. Kafka se despliega en una EC2 de AWS con Ansible y los servicios se conectan a él con la variable KAFKA_BOOTSTRAP. El build corre 38 tests en verde, las pruebas de punta a punta pasan por el gateway y los tres jobs batch terminan COMPLETED. Las salidas de terminal y capturas están en la carpeta evidencias/ del repositorio.

2. Contexto, procesos clave y requerimientos

Según el caso, el sistema legacy tiene cinco problemas: batch en mainframe sin reintentos ni reanudación; un backend monolítico que entrega la misma respuesta a web, móvil y cajero; ninguna tolerancia a fallos, porque un módulo caído tumba todo; seguridad centralizada sin separar canales; y escalado sólo vertical. De ahí salen los cinco procesos clave de la migración:

Tabla 1: Procesos clave de la migración

Proceso	Dónde quedó implementado
1. Batch a Spring Batch	Módulo batch: transaccionesJob, interesesJob y estadoCuentaJob con particionado, skip/retry y reejecución
2. Monolito a microservicios	ms-cuentas, ms-pagos y ms-clientes, cada uno con su schema, en Eureka y con su config en el config-server
3. Patrón BFF	bff-web, bff-mobile y bff-atm detrás del api-gateway HTTPS
4. Seguridad distribuida (Spring Security / OAuth2)	auth-server con un cliente y scopes por canal, JWT validado en cada servicio y client_credentials entre servicios
5. Mensajería asíncrona con Kafka	banco.transacciones y banco.alertas-seguridad: productor con outbox en ms-pagos, consumidor en notificaciones

Tabla 2: Requerimientos clave del negocio

Requerimiento	Cómo se resuelve
RN1. No perder ni duplicar dinero	Saga con compensación, Idempotency-Key obligatoria, débito/crédito idempotentes, outbox
RN2. Seguir atendiendo si falla una parte	Servicios independientes, Resilience4j con fallback, réplicas con Eureka y LoadBalancer
RN3. Cada canal con sus datos y permisos	Un BFF y un cliente OAuth2 por canal, scopes que limitan cada operación
RN4. Batch confiable y trazable	Skip/retry, cada fila descartada registrada, umbral de omisión y reejecución automática

3. Propuesta de arquitectura

Los canales entran por el api-gateway (Spring Cloud Gateway) en HTTPS, que enruta con lb:// al BFF de su canal y también pasa /oauth2 y /login al auth-server. Cada BFF valida el JWT, arma la respuesta de su canal y llama a los microservicios por Eureka. ms-cuentas es el único dueño del saldo; ms-pagos orquesta las operaciones de dinero sobre él y publica los eventos en Kafka. Hay un solo MySQL, pero con un schema y un usuario por servicio, así que ninguno puede leer las tablas de otro. La configuración de todos vive en config-repo/ y el batch corre aparte como contenedor de una sola ejecución.

Tabla 3: Decisiones de arquitectura y alternativas descartadas

Decisión	Elegido	Descartado y por qué
Descubrimiento	Eureka	Consul (otro componente que operar), DNS de Kubernetes o Cloud Map (amarran a una plataforma que no usamos). Eureka se integra directo con lb://
Configuración	Spring Cloud Config native	Variables sueltas por servicio: la config común (Resilience4j, issuer) se repetiría en cada uno
Mensajería	Kafka	RabbitMQ/JMS: borran el mensaje al consumirlo; Kafka retiene (7 y 30 días), permite releer y ordena por cuenta
Datos	Un schema por servicio	Base compartida: acopla los servicios por el esquema y no se despliegan por separado
Consistencia	Saga + outbox	2PC/XA: bloquea a todos los participantes y Kafka no participa en XA
Entrada y balanceo	Gateway + LoadBalancer	Balanceador de AWS (ALB): no hace falta, el gateway reparte entre las réplicas que hay en Eureka
Seguridad	OAuth2 + JWT	Sesiones o API keys: el JWT se valida con la llave pública sin llamar al auth-server cada vez
Resiliencia	Resilience4j	Hystrix: está en mantenimiento y no funciona con Spring Boot 3
Batch	Particionado	Step multi-hilo: FlatFileItemReader no es thread-safe y el número de línea dejaría de ser correcto

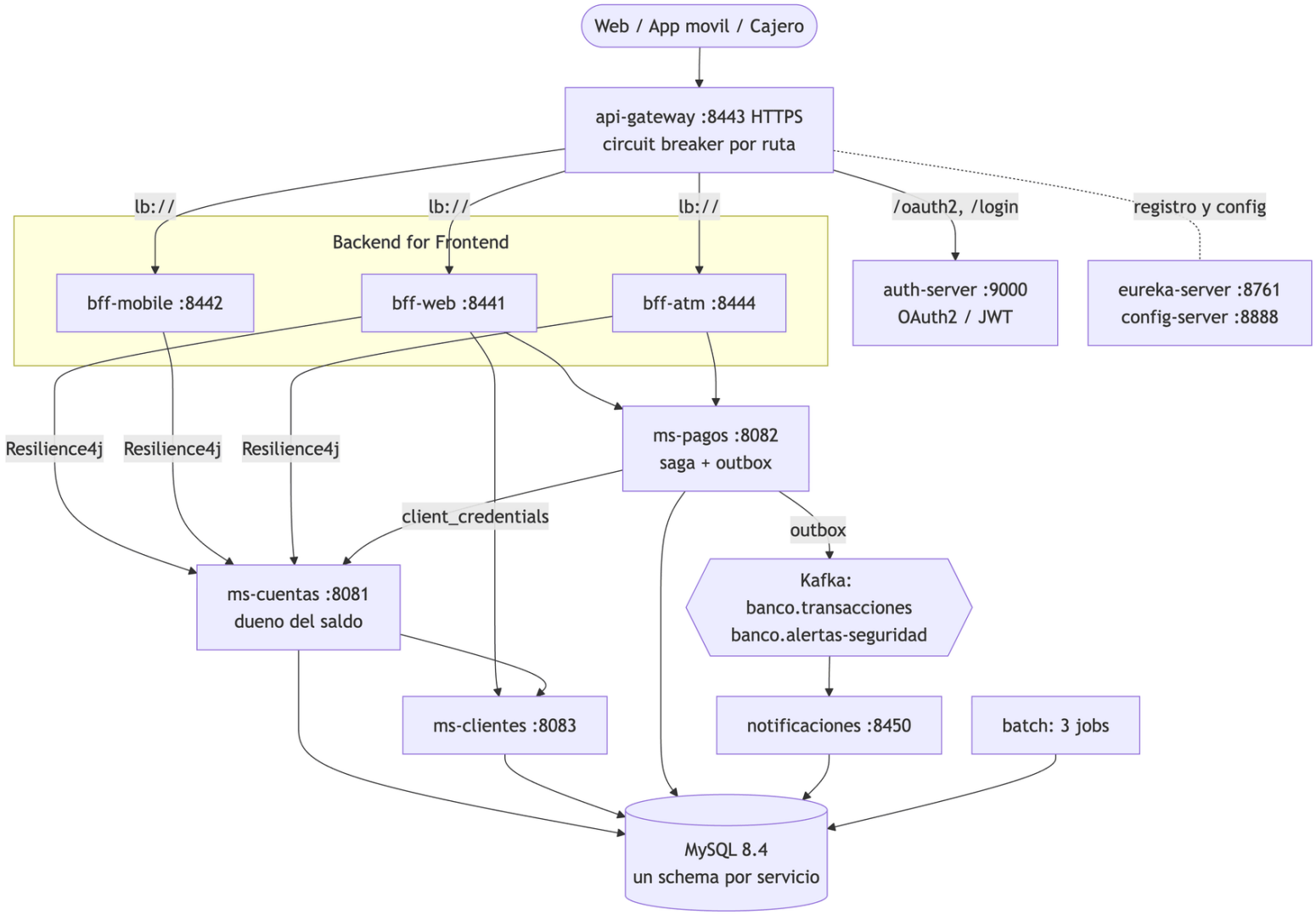


Figura 1: Arquitectura general

4. Parte 1: procesos batch con Spring Batch

Los datos vienen de KariVillagran/fin_legacy_data (Finanzas ABC): movimientos_financieros_diarios.csv, intereses_trimestrales.csv y estados_financieros_anuales.csv, con 1000 filas cada uno en semana_3. Es un repositorio distinto al de la sumativa 1, pero con el mismo esquema, así que reutilicé los componentes de esa entrega (lectores particionados, normalizers, política de skip, reportes de omitidos) y los adapté: nombres de archivo, la semana como parámetro (batch.semana), el tipo "invalido" como dato ausente y la reejecución automática. Cada job se corre como contenedor con docker compose run batch --spring.batch.job.name=<job>.

- **Jobs:** transaccionesJob (reporte de transacciones diarias, totales de crédito y débito), interesesJob (ahorro 5 %, préstamo 10 %; hipoteca queda fuera de alcance) y estadoCuentaJob (estados de cuenta anuales, más un paso de agregación por cuenta y año y un reporte de auditoría).
- **Errores de datos:** una fila ilegible se salta con InvalidDataSkipPolicy y una inválida la filtra el processor. Las dos quedan en la tabla omitidos y en un CSV, con la línea y el motivo.
- **Fallos temporales:** una TransientDataAccessException (caída de conexión, deadlock) reintenta el chunk hasta 3 veces con backoff exponencial.
- **Volumen:** CsvPartitioner reparte el archivo en rangos de líneas que se procesan en paralelo (grid = mitad de los núcleos, chunk 100) y el writer usa INSERT por lotes.
- **Finalización:** si un job descarta más que su umbral (70 %, 85 % y 45 %) termina FAILED y el contenedor sale con código 1. Si un step falla por un error crítico, ReejecucionRunner relanza la misma instancia hasta 3 veces, y Spring Batch sigue desde la partición que falló.

Tabla 4: Resultado de los jobs (evidencias/04-batch.txt)

Job	Origen	Escritas	Descartadas	Tasa	Umbral	Estado
transaccionesJob	1000	392	608	60,8 %	70 %	COMPLETED
interesesJob	1000	375	625	62,5 %	85 %	COMPLETED
estadoCuentaJob	1000	604	396	39,6 %	45 %	COMPLETED

Escritas más descartadas suman el origen en los tres jobs y los conteos coinciden con las tablas de MySQL. Las tasas son altas porque el dataset trae a propósito tipos "invalido", montos vacíos o negativos y fechas imposibles, y esos datos no se pueden inventar. La evidencia también muestra la reejecución ante una falla crítica en transaccionesJob: la ejecución 4 quedó FAILED con una partición completa (204 filas escritas) y otra caída en su primer chunk. ReejecucionRunner la relanzó a los 2000 ms como la misma instancia, que corrió sólo la partición que faltaba (188 filas) y terminó COMPLETED, con 392 filas en total, las mismas que la corrida normal (392).

5. Parte 2: Backend for Frontend

Cada canal tiene su BFF, con su propio módulo, imagen y configuración, así cada equipo de frontend puede cambiar su API sin tocar a los otros. El gateway sólo enruta por prefijo.

Tabla 5: Los tres BFF

BFF	Endpoints	Scopes	Qué optimiza
bff-web :8441	/api/web/me/dashboard, movimientos, operaciones, transferencias, pagos, apertura y cierre de cuenta, perfil	web:read, web:write	Datos completos para una interfaz compleja
bff-mobile :8442	/api/mobile/me/inicio	mobile:read	Una sola llamada con nombre, cuentas enmascaradas y últimos movimientos
bff-atm :8444	/api/atm/cuentas/{id}/saldo, retiros, depositos	atm:balance, atm:withdraw, atm:deposit	Respuestas mínimas, retiro hasta 200.000 y sin reintentos de dinero

En el auth-server hay un cliente OAuth2 por canal (bank-web, bank-mobile, bank-atm) con authorization code + PKCE, y cada uno sólo puede pedir los scopes de su canal. Cada BFF exige su audiencia y los microservicios vuelven a revisar el scope con @PreAuthorize. En las pruebas, una llamada sin token y un token del móvil en el cajero dan 401, y el token de otro titular sobre la cuenta 101 da 403. El gateway publica sólo el 8443 con TLS (keystore PKCS12 autofirmado que se genera al construir la imagen); los BFF y microservicios no publican puertos y el login también pasa por el gateway. El flujo OAuth2 completo está en el Anexo B.

6. Parte 3: microservicios con Spring Cloud

Tabla 6: Microservicios

Servicio	Responsabilidad	Datos propios
ms-cuentas :8081	Apertura, cierre, mantenimiento y saldo; débito, crédito y reversa internos (scope cuentas:operar)	account, movement, operacion_aplicada
ms-pagos :8082	Retiros, depósitos, transferencias y pago de servicios; orquesta la saga	operacion, evento_pendiente (outbox)
ms-clientes :8083	Datos personales y perfil	customer_profile
notificaciones :8450	Consume los eventos de Kafka	notificacion

6.1 Config, Eureka, balanceo y seguridad

El config-server (8888, perfil native, con usuario y clave) entrega a cada servicio su archivo de config-repo/ más un application.yml común; los servicios sólo tienen su nombre y puerto. Todos se registran en eureka-server (8761) y se llaman por nombre con un RestClient @LoadBalanced (http://ms-cuentas). El gateway enruta /api/web, /api/mobile y /api/atm a lb://bff-*, y cada ruta tiene su circuit breaker con un fallback 503. Todos son resource servers que validan el JWT; cuando ms-pagos llama a ms-cuentas usa su propio token client_credentials con el scope cuentas:operar, que ningún canal tiene.

6.2 Resilience4j y comportamientos alternativos

La configuración está una sola vez en config-repo/application.yml. Circuit breaker: ventana de 10 llamadas, mínimo 4, abre con 50 % de fallas, queda abierto 15 s y en half-open deja pasar 2 llamadas de prueba; los 4xx no cuentan como falla. Retry: 3 intentos cada 100 ms, sólo si no hay conexión o hay un 5xx. El circuit breaker va afuera y el retry adentro, así los reintentos cuentan como una sola falla y el fallback corre al final. Los POST de dinero desde los BFF sólo llevan circuit breaker, para no duplicar un cobro.

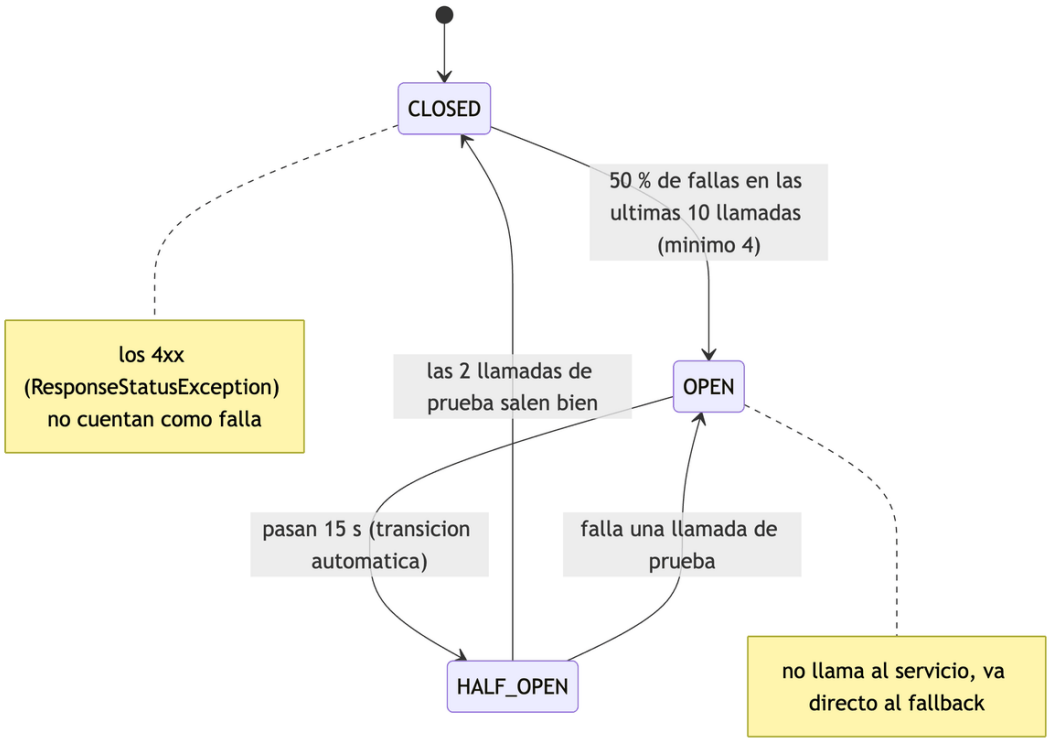


Figura 2: Estados del circuit breaker con la configuración del proyecto

Tabla 7: Dónde se aplica y qué hace el fallback

Llamada	Retry	Fallback
gateway → BFF	No	503 con degraded=true
bff-web y bff-mobile → ms-cuentas (lecturas)	Sí	Respuesta degradada con aviso, en vez de error
bff-atm → ms-cuentas (saldo)	Sí	503 "Cajero sin conexión con el banco"
BFF → ms-pagos (retiro, depósito, transferencia, pago)	No	503 sin cargo; un 4xx de negocio pasa tal cual
ms-pagos → ms-cuentas (débito, crédito, reversa)	Sí, son idempotentes	La saga compensa
ms-cuentas → ms-clientes (validar cliente al abrir cuenta)	Sí	503 "No se pudo validar el cliente"

6.3 Kafka y consistencia de datos

Tabla 8: Tópicos

Tópico	Part. / retención	Productor → consumidor	Evento
banco.transacciones	3 / 7 días	ms-pagos (outbox) → notificaciones	TransaccionCompletada (retiro, depósito, transferencia, pago)
banco.alertas-seguridad	1 / 30 días	ms-pagos (outbox) → notificaciones	AlertaSeguridad: monto desde 150.000 o 3 rechazos en 10 min

No hay una transacción que cubra las dos cuentas, la operación y el evento. ms-pagos guarda la operación PENDIENTE, pide el débito y el crédito (cada pierna con su operationId, idempotente) y, si el crédito falla, revierte primero el crédito (si alcanzó a quedar) y después el débito. Si la reversa no se puede hacer, un reconciliador la reintenta cada 15 s y después de 5 intentos la deja en REQUIERE_REVISION. Una transferencia entre cuentas de distinta moneda se rechaza antes de mover plata. El estado COMPLETADA y el evento se guardan en la misma transacción local (outbox) y se publican cada 5 s con key = cuenta; notificaciones confirma el offset a mano, descarta repetidos por el id del evento y, si no puede guardar uno, lo reintenta con espera y al final lo manda a un tópico .DLT. Como monitoreo, todos exponen actuator (health, metrics, circuitbreakers) y los logs llevan traceId para seguir una operación entre servicios.

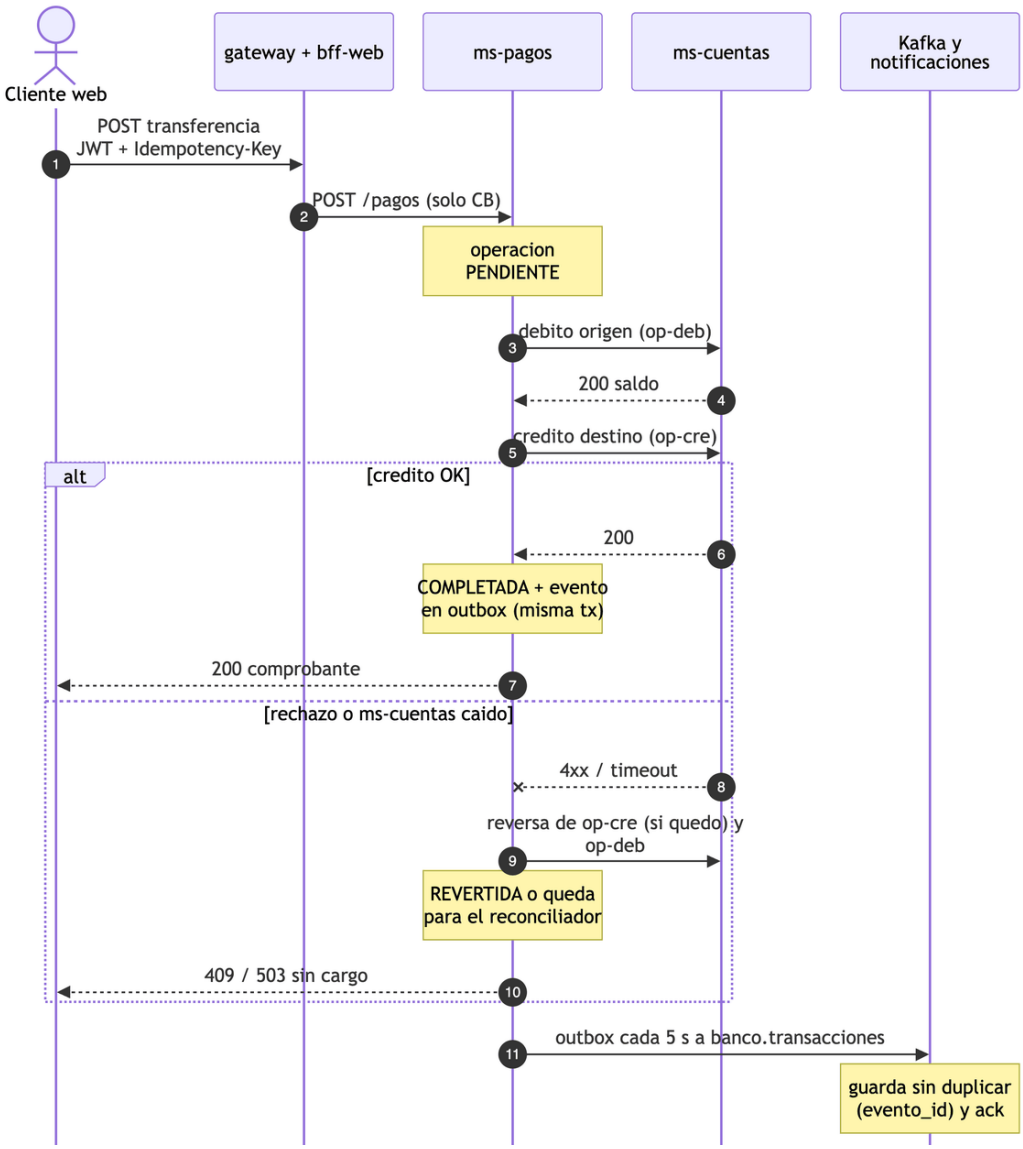


Figura 3: Transferencia con compensación y evento por outbox

7. Docker, escalado horizontal y despliegue en AWS

Cada módulo tiene un Dockerfile multi-etapa y el docker-compose.yaml levanta MySQL, config, eureka, gateway, auth, los tres BFF, los tres microservicios y notificaciones, y con el profile kafka-local también Kafka y kafka-ui. Para que cupiera en 4 CPU y 8 GB cada JVM va con heap

de 200 MB y serial GC. Con todo arriba y dos réplicas de ms-cuentas, 15 contenedores usan 2887 MiB.

Los microservicios, BFF y notificaciones no tienen container_name ni puerto publicado, así que se escalan con docker compose up -d --scale ms-cuentas=2. Eureka mostró 2 instancias de ms-cuentas y las 10 consultas de saldo atendidas se repartieron 5 y 5 entre ellas.

En AWS (Learner Lab) se despliega Kafka con Ansible, en infra/ansible:

- **provision.yml:** EC2 t3.medium con Amazon Linux 2023 (AMI desde SSM) y 20 GB, y un security group con 22, 9094 y 8090 abiertos sólo a mi IP.
- **deploy.yml:** instala Docker con los mismos comandos de la clase 7 (dnf install docker, habilitar el servicio, ec2-user al grupo docker) más el plugin de compose; levanta un broker KRaft apache/kafka:3.9.1 (heap 512 MB, listener externo 9094 con la IP pública) y kafka-ui en el 8090; crea los dos tópicos con factor de replicación 1 e imprime KAFKA_BOOTSTRAP=<ip>:9094.
- **teardown.yml:** borra todo al terminar.

En local se quita el profile kafka-local del .env y se pone KAFKA_BOOTSTRAP=<ip pública>:9094. No hay balanceador de AWS: el balanceo es el del gateway y LoadBalancer con Eureka. Si la EC2 se reinicia cambia la IP y hay que volver a correr provision.yml y deploy.yml.

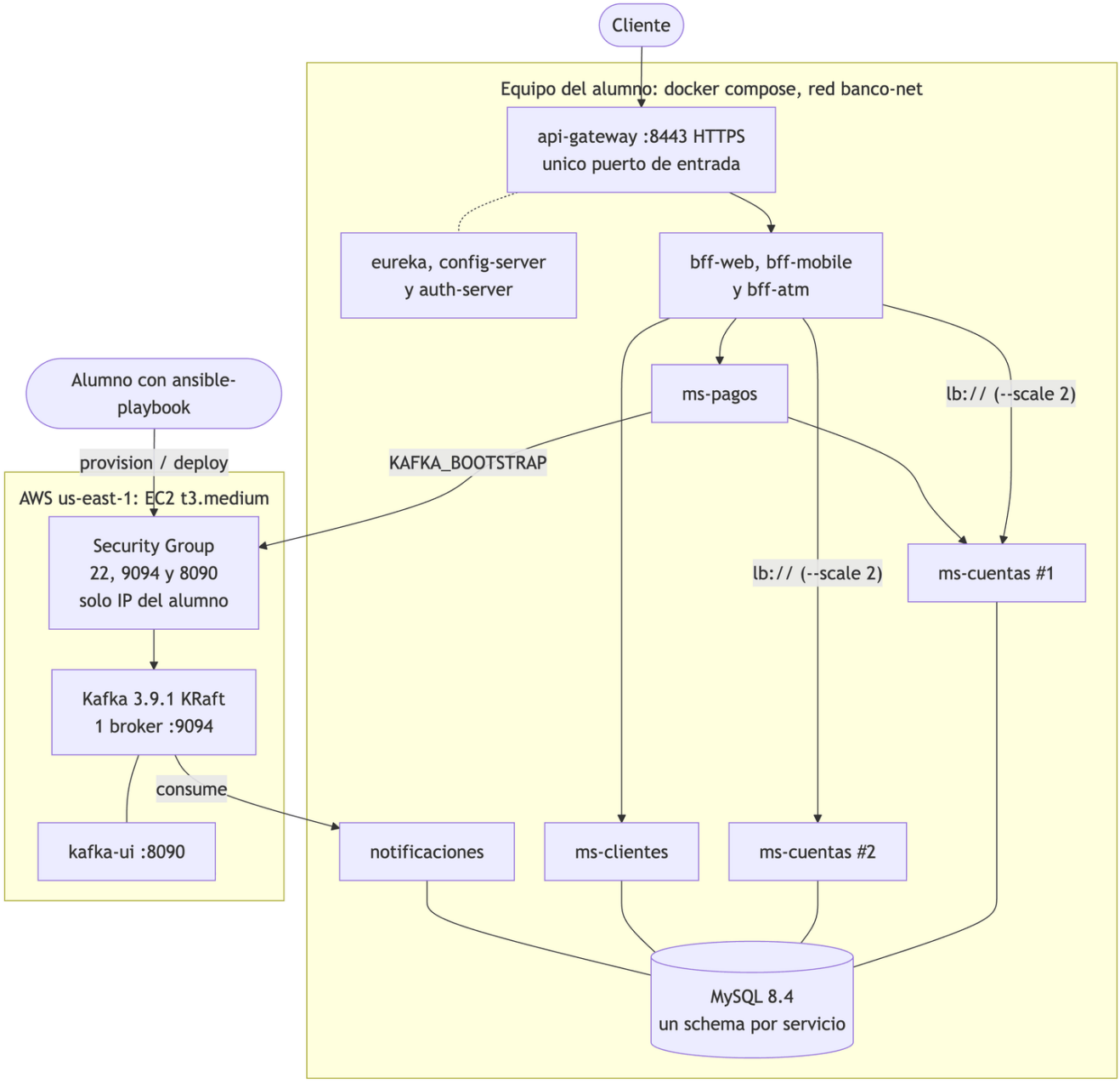


Figura 4: Despliegue: servicios en Docker Compose y Kafka en una EC2 de AWS

7.1 Verificación de Kafka en EC2

En la sumativa 3 la mensajería quedó con observación porque Kafka sólo corría en local. En esta entrega Kafka se despliega en la EC2 con los playbooks provision.yml y deploy.yml. Para verificarlo: la instancia debe aparecer running en la consola de EC2 con su IP y DNS públicos; por SSH, docker compose ps dentro de la instancia muestra kafka y kafka-ui arriba y kafka-topics.sh --describe muestra banco.transacciones y banco.alertas-seguridad en el broker; en

local, ms-pagos y notificaciones se levantan con KAFKA_BOOTSTRAP=<ip pública>:9094, y una operación hecha por el gateway genera un evento que llega a notificaciones (GET /api/notificaciones) y se ve en kafka-ui en el puerto 8090.

8. Resultados y comparación con el sistema legacy

- **Build:** mvn package en verde, 38 tests en 12 clases de test de los 13 módulos.
- **Punta a punta:** tokens por canal, los tres BFF por el gateway, retiro, depósito, transferencia y pago; la misma Idempotency-Key devuelve el mismo comprobante sin descontar dos veces; una transferencia a una cuenta inexistente devuelve 409 y el dinero vuelve a la cuenta de origen; los eventos llegan a notificaciones.
- **Resiliencia:** con ms-cuentas detenido, /actuator/retryevents muestra cada llamada con los intentos 1 y 2 (RETRY) y el 3 (ERROR) por connect timeout. La web responde degradada en 1,7 s y la transferencia da 503 sin hacer cambios. Cuando el circuito se abre (50 % de fallas en 6 llamadas) el fallback responde en 0,017 s y el cajero da 503. Al volver ms-cuentas el circuito pasa a HALF_OPEN y luego a CLOSED.
- **Batch:** los tres jobs COMPLETED (Tabla 4) y la reejecución automática probada.

Tabla 9: Legacy y nuevo sistema

Aspecto	Legacy	Nuevo
Arquitectura	Monolito en mainframe	3 microservicios, 3 BFF y gateway, desplegables por separado
Canales	Misma respuesta para todos	Un BFF por canal
Escalado	Vertical	Horizontal por servicio con --scale y Eureka
Fallas	Un módulo caído afecta a todo	Circuit breaker, retry y respuestas degradadas
Seguridad	Centralizada	OAuth2/JWT con scopes por canal, HTTPS y token de servicio
Dinero	Sin protección ante reintentos	Idempotencia, saga con compensación y reconciliador
Batch	Scripts sin reintento	Skip/retry, particiones, umbral y reejecución; cada descarte con su motivo

9. Desafíos y soluciones

Tabla 10: Desafíos

Desafío	Solución
Separar el bank-api de la semana 8 sin perder consistencia del saldo	ms-cuentas como único dueño del saldo y saga con compensación en ms-pagos
ms-cuentas se cae entre el débito y el crédito	Reversa de las dos piernas y reconciliador programado
Reintentar sin duplicar dinero	Retry sólo en lecturas y llamadas idempotentes; Idempotency-Key en las escrituras
Eventos perdidos si Kafka no está	Outbox en la base de ms-pagos; el evento sale cuando Kafka vuelve
En la sumativa 3 Kafka sólo corría local	Ahora se despliega en EC2 con Ansible y se evidencia la conexión remota
Todo en 4 CPU y 8 GB	Heap de 200 MB por JVM y un broker de Kafka
HTTPS en un clone limpio	Keystore PKCS12 generado al construir la imagen del gateway

10. Mejoras y próximos pasos

- **Todo en AWS:** imágenes en ECR, servicios en ECS Fargate con auto scaling, Amazon MSK (3 brokers), RDS MySQL por servicio, ALB con certificado de ACM, Secrets Manager y CloudWatch Logs.
- **Auth-server:** usuarios en base de datos y llave de firma persistente (hoy se genera al arrancar).
- **Observabilidad:** Prometheus/Grafana para métricas y Zipkin para ver las trazas que ya se generan.
- **Kafka:** dead letter topic y un consumidor antifraude para las alertas.
- **Batch:** programar los jobs (EventBridge o cron) y dejar los reportes en S3.

11. Anexos

A. Repositorio y documentación

Repositorio de la entrega: https://github.com/Dalvae/banco-xyz-eft. Ahí están readme.md (resumen, servicios y puertos, cómo se cubre cada requisito), instrucciones.md (cómo ejecutar y probar cada componente con curl), despliegue.md (despliegue en AWS), docs/arquitectura.md (los diagramas en Mermaid) y evidencias/ (salidas de terminal y capturas). Las entregas anteriores son sólo antecedentes: el batch viene del repositorio banco-xyz-batch de la sumativa 1, y los BFF y el auth-server de las entregas anteriores de las semanas 5 y 8. Todo eso se copió y adaptó dentro de banco-xyz-eft.

B. Diagramas complementarios

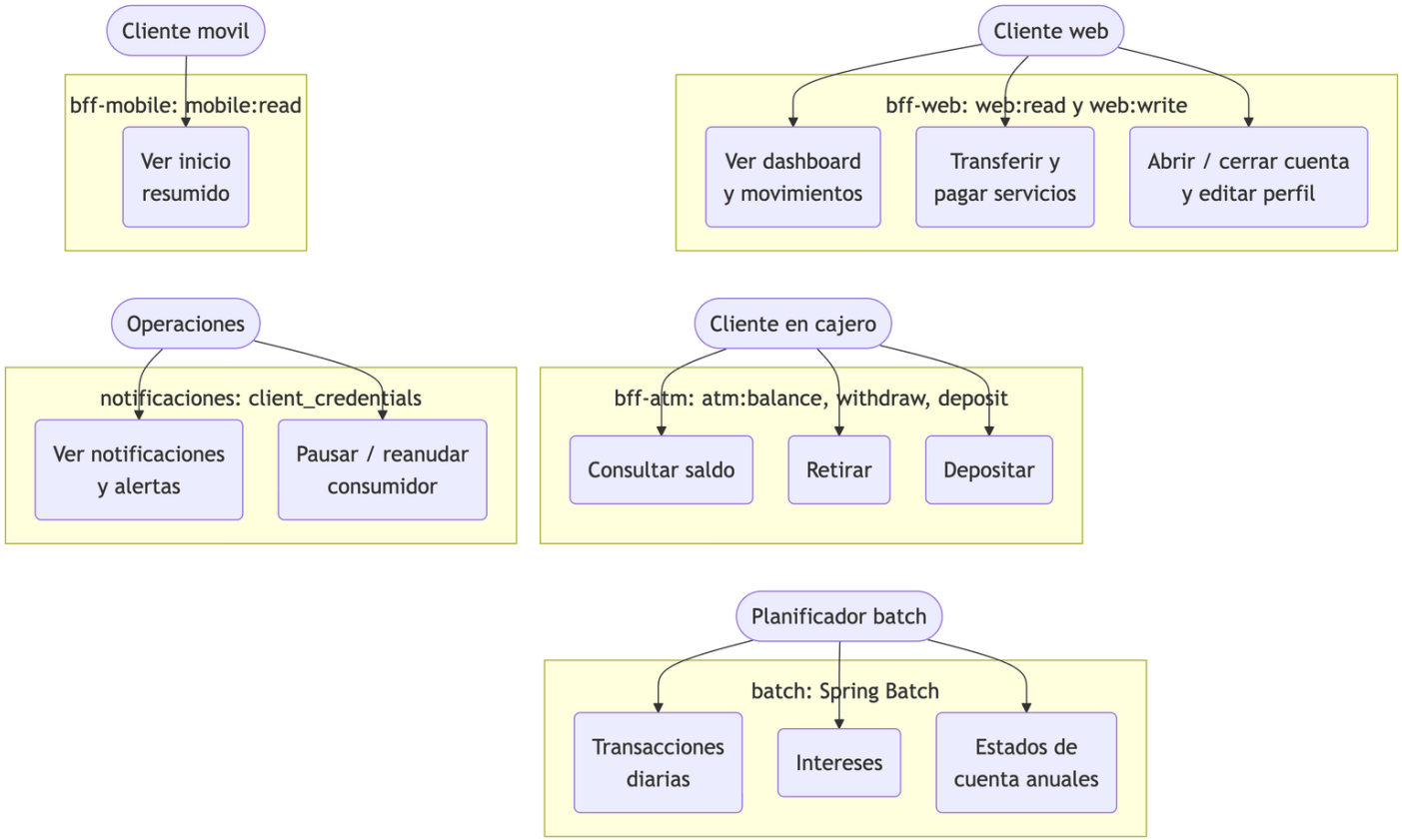


Figura B1: Casos de uso por canal

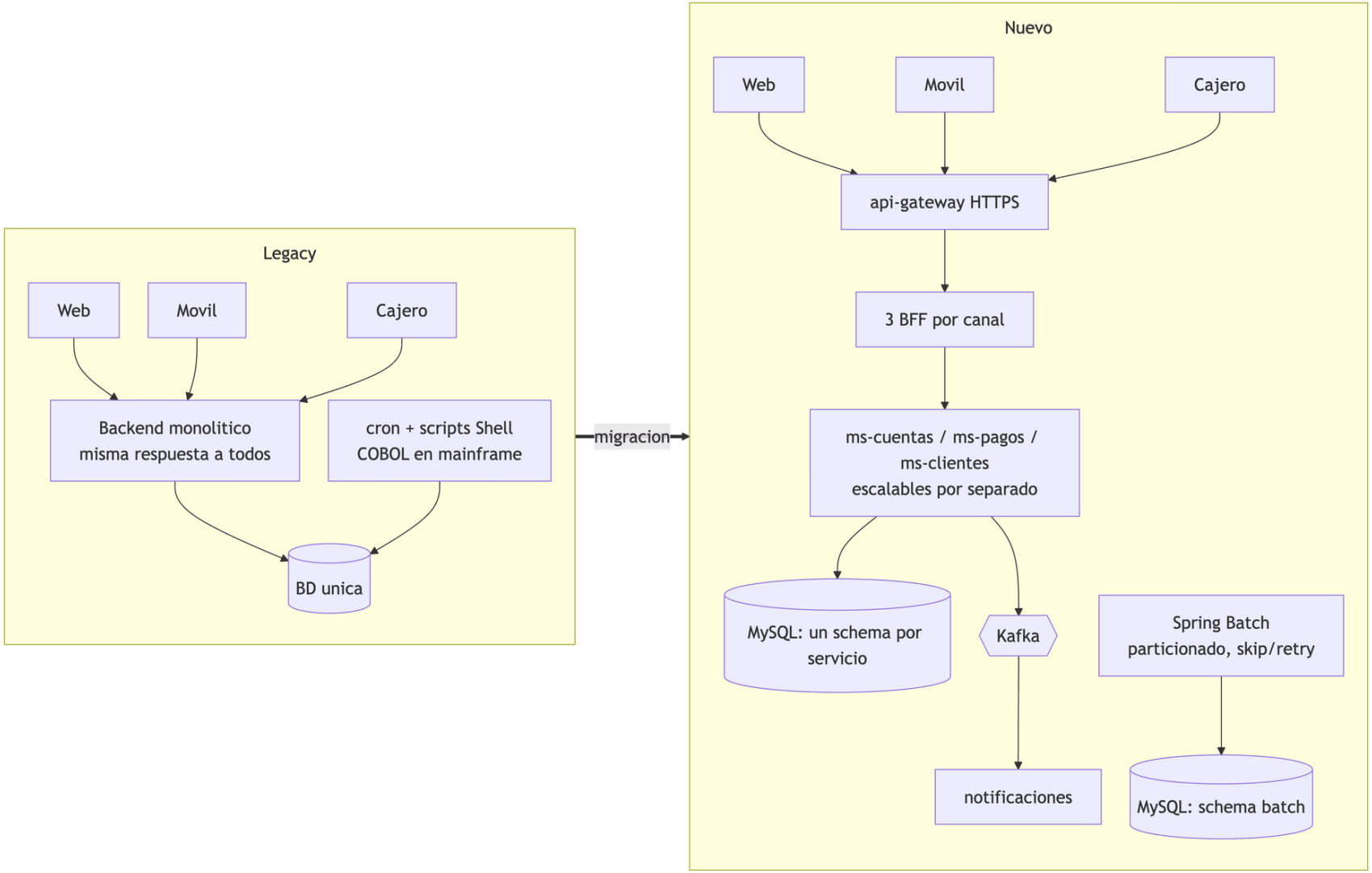


Figura B2: Sistema legacy y sistema nuevo

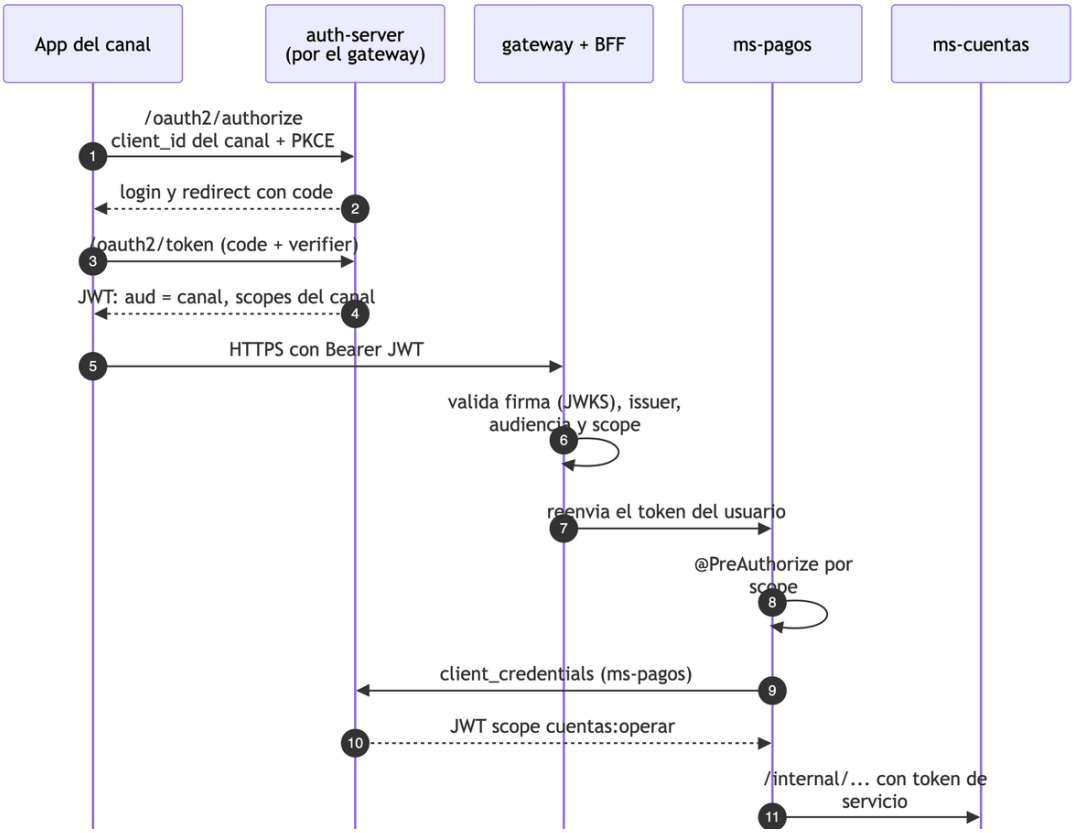


Figura B3: Flujo OAuth2 por canal y token de servicio

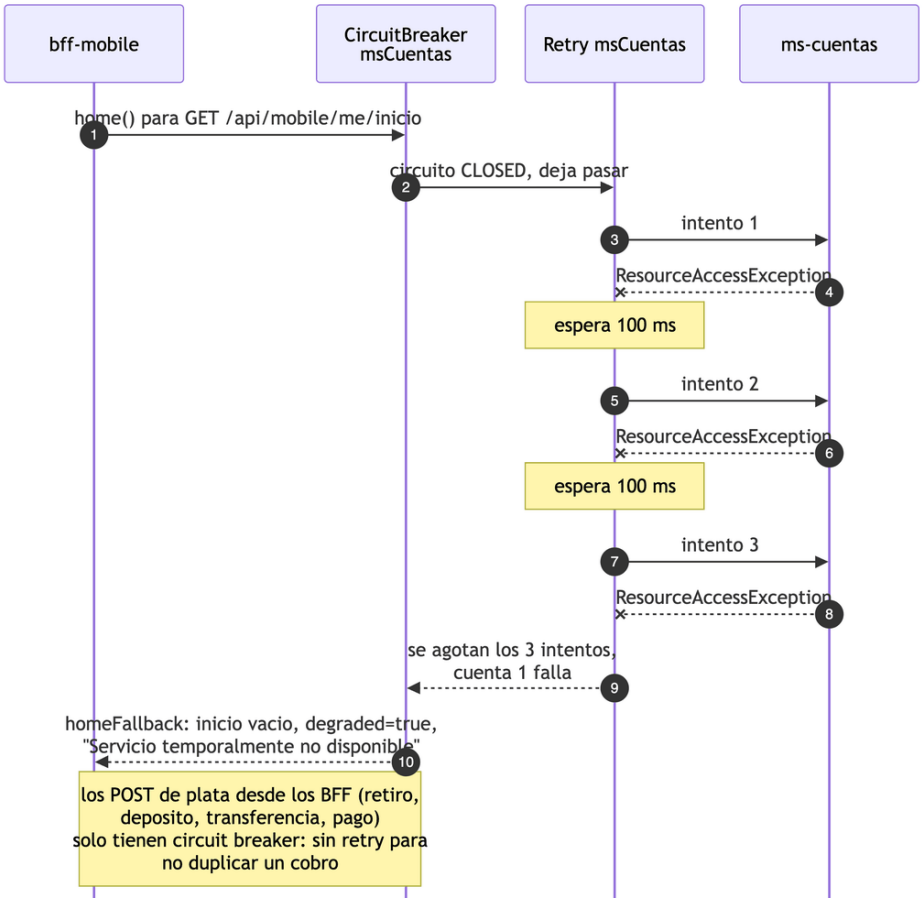


Figura B4: Circuit breaker afuera y retry adentro

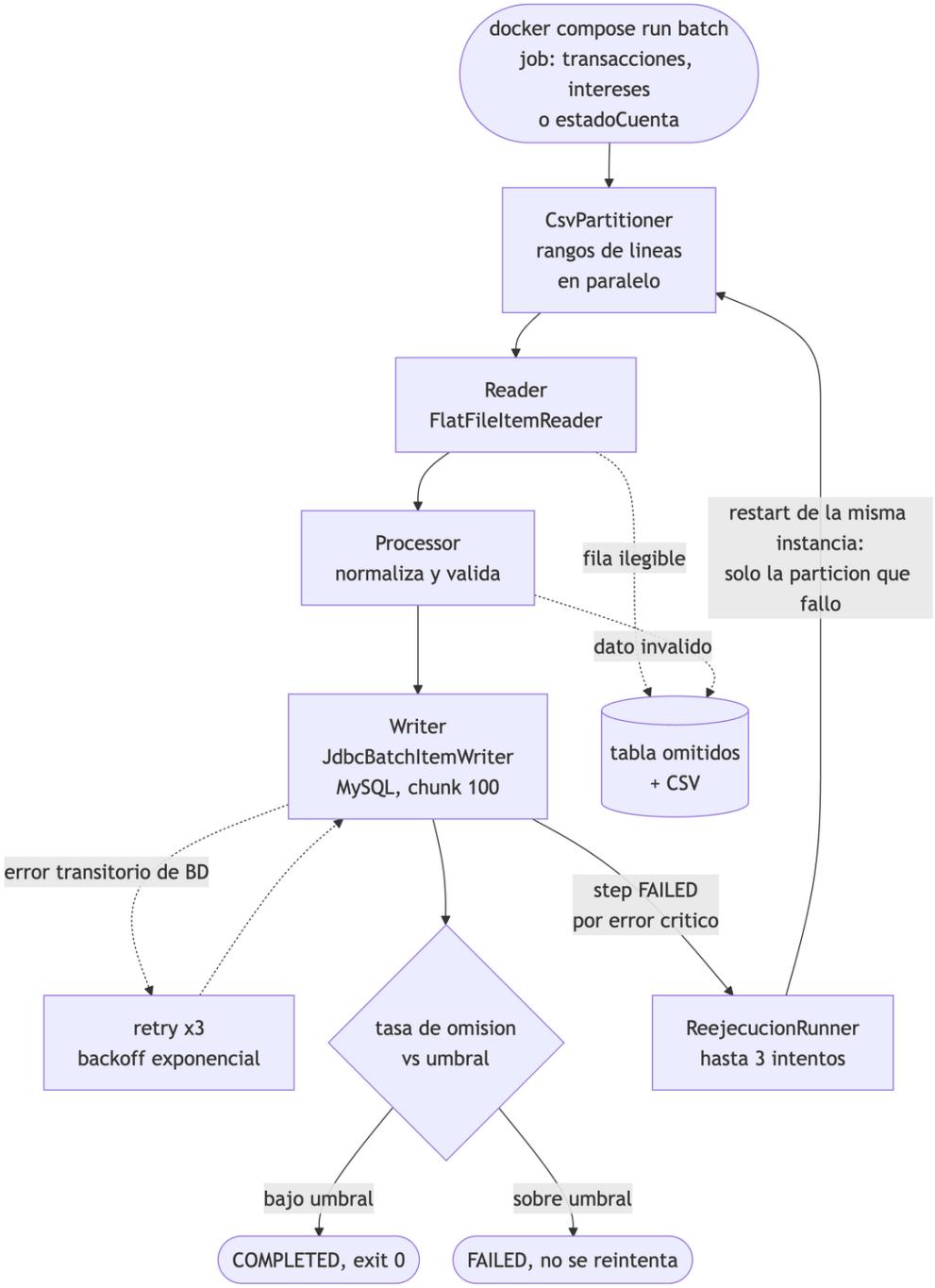
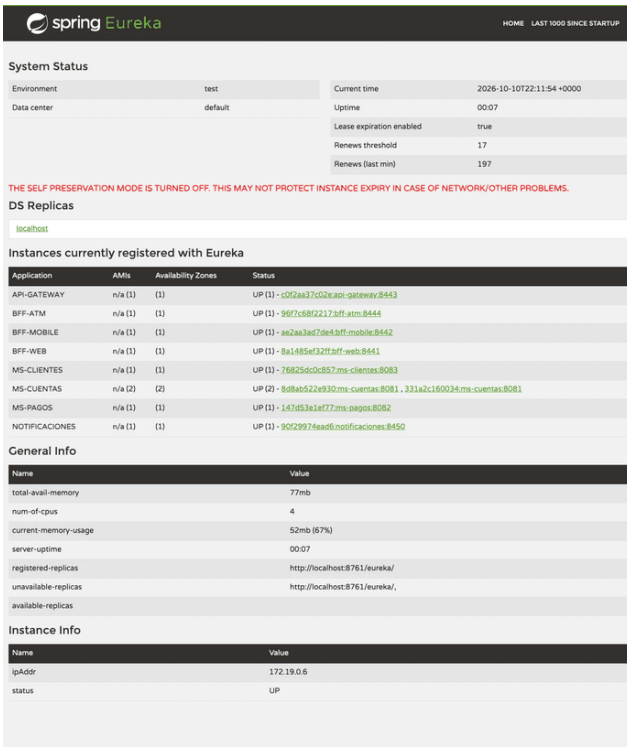


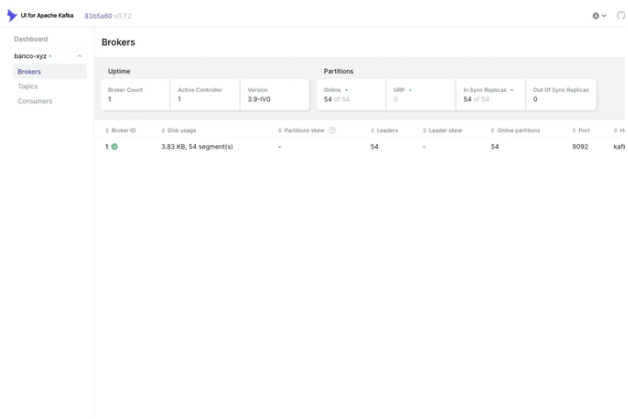
Figura B5: Flujo de los jobs batch

C. Capturas

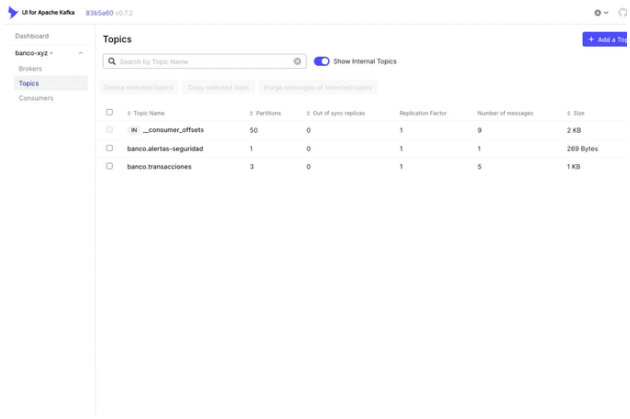
Capturas de evidencias/capturas/. Las de terminal, que son anchas, van a todo el ancho para que se lean.



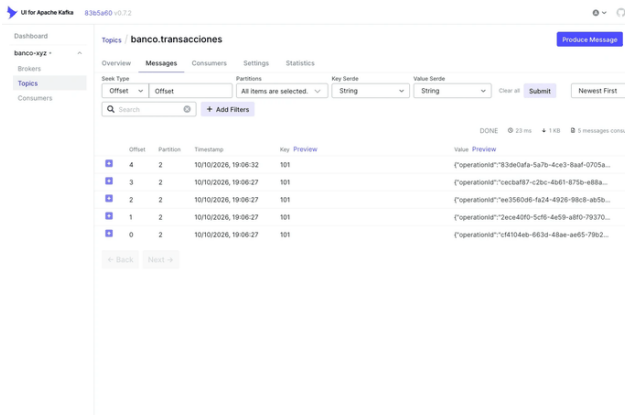
Captura 1: 01-eureka-ms-cuentas-2-instancias



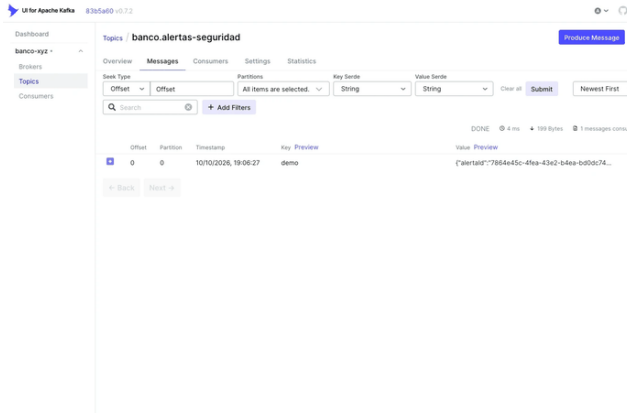
Captura 2: 02-kafka-ui-broker



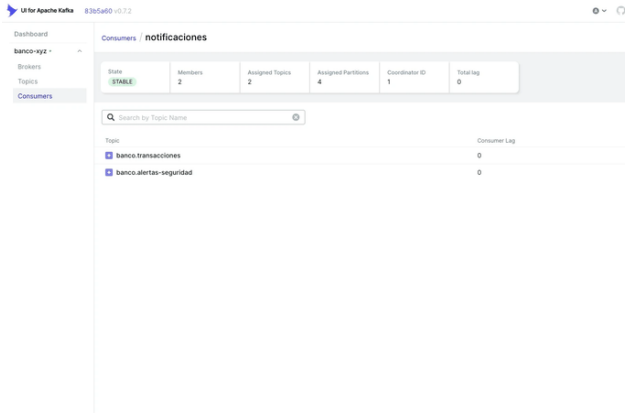
Captura 3: 03-kafka-ui-topicos



Captura 4: 04-kafka-ui-mensajes-transacciones



Captura 5: 05-kafka-ui-mensajes-alertas



Captura 6: 06-kafka-ui-consumidor

```
~/duoc/backend3/repo/s9  main ± curl -s -u config:$CONFIG_PASSWORD localhost:8888/api-gateway/default | jq '.p
rtySources[0].source'
{
  "spring.cloud.gateway.server.webflux.trusted-proxies": "(10|127|172|192\\.168)\\..*",
  "spring.cloud.gateway.server.webflux.routes[0].id": "bff-web",
  "spring.cloud.gateway.server.webflux.routes[0].uri": "lb://bff-web",
  "spring.cloud.gateway.server.webflux.routes[0].predicates[0]": "Path=/api/web/**",
  "spring.cloud.gateway.server.webflux.routes[0].filters[0].name": "CircuitBreaker",
  "spring.cloud.gateway.server.webflux.routes[0].filters[0].args.name": "bffWeb",
  "spring.cloud.gateway.server.webflux.routes[0].filters[0].args.fallbackUri": "forward:/fallback/web",
  "spring.cloud.gateway.server.webflux.routes[1].id": "bff-mobile",
  "spring.cloud.gateway.server.webflux.routes[1].uri": "lb://bff-mobile",
  "spring.cloud.gateway.server.webflux.routes[1].predicates[0]": "Path=/api/mobile/**",
  "spring.cloud.gateway.server.webflux.routes[1].filters[0].name": "CircuitBreaker",
  "spring.cloud.gateway.server.webflux.routes[1].filters[0].args.name": "bffMobile",
  "spring.cloud.gateway.server.webflux.routes[1].filters[0].args.fallbackUri": "forward:/fallback/mobile",
  "spring.cloud.gateway.server.webflux.routes[2].id": "bff-atm",
  "spring.cloud.gateway.server.webflux.routes[2].uri": "lb://bff-atm",
  "spring.cloud.gateway.server.webflux.routes[2].predicates[0]": "Path=/api/atm/**",
  "spring.cloud.gateway.server.webflux.routes[2].filters[0].name": "CircuitBreaker",
  "spring.cloud.gateway.server.webflux.routes[2].filters[0].args.name": "bffAtm",
  "spring.cloud.gateway.server.webflux.routes[2].filters[0].args.fallbackUri": "forward:/fallback/atm",
  "spring.cloud.gateway.server.webflux.routes[3].id": "notificaciones",
  "spring.cloud.gateway.server.webflux.routes[3].uri": "lb://notificaciones",
  "spring.cloud.gateway.server.webflux.routes[3].predicates[0]": "Path=/api/notificaciones/**",
  "spring.cloud.gateway.server.webflux.routes[4].id": "auth-server",
  "spring.cloud.gateway.server.webflux.routes[4].uri": "${AUTH_URL:http://localhost:9000}",
  "spring.cloud.gateway.server.webflux.routes[4].predicates[0]": "Path=/oauth2/**,/login,/.well-known/**,/userinfo",
  "resilience4j.circuitbreaker.configs.default.sliding-window-size": 10,
  "resilience4j.circuitbreaker.configs.default.minimum-number-of-calls": 4,
  "resilience4j.circuitbreaker.configs.default.failure-rate-threshold": 50,
  "resilience4j.circuitbreaker.configs.default.wait-duration-in-open-state": "15s",
  "resilience4j.timelimiter.configs.default.timeout-duration": "10s",
  "management.endpoints.web.exposure.include": "health,info,gateway,circuitbreakers",
  "management.endpoint.gateway.access": "read-only"
}
~/duoc/backend3/repo/s9  main ±
```

Captura 8: 08-config-server-api-gateway

```
~/duoc/backend3/repo/s9  main ±  curl -s -u config:$CONFIG_PASSWORD localhost:8888/ms-pagos/default | jq '.property
Sources[0].source'
{
  "spring.datasource.url": "jdbc:mysql://${MYSQL_HOST:localhost}:3306/pagos?useSSL=false&allowPublicKeyRetrieval=true&se
rverTimezone=UTC",
  "spring.datasource.username": "pagos",
  "spring.datasource.password": "${DB_PASSWORD:banco}",
  "spring.sql.init.mode": "always",
  "spring.kafka.bootstrap-servers": "${KAFKA_BOOTSTRAP:localhost:19092}",
  "spring.security.oauth2.resourceserver.jwt.audiences": "bank-web,bank-mobile,bank-atm",
  "spring.security.oauth2.client.registration.ms-pagos.client-id": "ms-pagos",
  "spring.security.oauth2.client.registration.ms-pagos.client-secret": "${PAGOS_CLIENT_SECRET:cambiar}",
  "spring.security.oauth2.client.registration.ms-pagos.authorization-grant-type": "client_credentials",
  "spring.security.oauth2.client.registration.ms-pagos.scope": "cuentas:operar",
  "spring.security.oauth2.client.provider.ms-pagos.token-uri": "${AUTH_URL:http://localhost:9000}/oauth2/token",
  "banco.kafka.replicas": "${KAFKA_REPLICAS:1}",
  "pagos.alertas.monto-alto": 150000,
  "pagos.alertas.intentos-fallidos": 3,
  "pagos.reconciliacion-ms": 15000,
  "eventos.reintento-ms": 5000
}
~/duoc/backend3/repo/s9  main ±
```

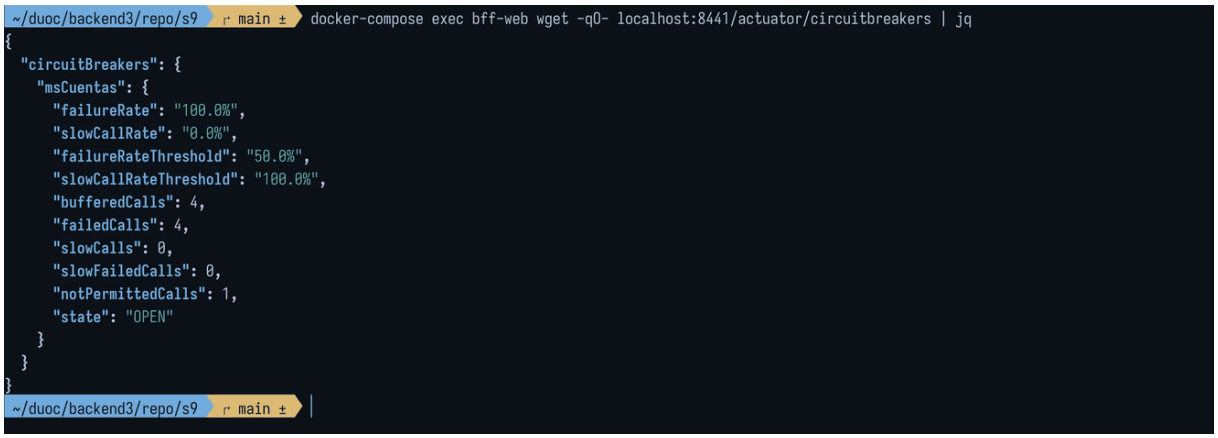
Captura 7: 07-config-server-ms-pagos

```
~/duoc/backend3/repo/s9  main ±  curl -sk $G/actuator/gateway/routes | jq -c '.[] | {route_id, uri}'
{"route_id":"bff-web","uri":"lb://bff-web"}
{"route_id":"bff-mobile","uri":"lb://bff-mobile"}
{"route_id":"bff-atm","uri":"lb://bff-atm"}
{"route_id":"notificaciones","uri":"lb://notificaciones"}
{"route_id":"auth-server","uri":"http://auth-server:9000"}
~/duoc/backend3/repo/s9  main ±
```

Captura 9: 09-gateway-rutas-https

```
~/duoc/backend3/repo/s9  main ±  docker-compose stop ms-cuentas
[+] stop 1/1
 ✓ Container s9-ms-cuentas-1 Stopped                                                                              3.0s
~/duoc/backend3/repo/s9  main ±  curl -sk -w '  %{http_code} %{time_total}s\n' -H "Authorization: Bearer $WEB" $G/api/web/me/dashboard
{"profile":null,"accounts":[],"degraded":true,"notice":"No pudimos conectar con el banco, intenta en unos segundos"}  200 1.703310s
~/duoc/backend3/repo/s9  main ±  curl -sk -w '  %{http_code} %{time_total}s\n' -H "Authorization: Bearer $WEB" $G/api/web/me/dashboard
{"profile":null,"accounts":[],"degraded":true,"notice":"No pudimos conectar con el banco, intenta en unos segundos"}  200 1.682305s
~/duoc/backend3/repo/s9  main ±  curl -sk -w '  %{http_code} %{time_total}s\n' -H "Authorization: Bearer $WEB" $G/api/web/me/dashboard
{"profile":null,"accounts":[],"degraded":true,"notice":"No pudimos conectar con el banco, intenta en unos segundos"}  200 1.706899s
~/duoc/backend3/repo/s9  main ±  curl -sk -w '  %{http_code} %{time_total}s\n' -H "Authorization: Bearer $WEB" $G/api/web/me/dashboard
{"profile":null,"accounts":[],"degraded":true,"notice":"No pudimos conectar con el banco, intenta en unos segundos"}  200 1.690639s
~/duoc/backend3/repo/s9  main ±  curl -sk -w '  %{http_code} %{time_total}s\n' -H "Authorization: Bearer $WEB" $G/api/web/me/dashboard
{"profile":null,"accounts":[],"degraded":true,"notice":"No pudimos conectar con el banco, intenta en unos segundos"}  200 0.017356s
~/duoc/backend3/repo/s9  main ±  curl -sk -w '  %{http_code}\n' -H "Authorization: Bearer $ATM" $G/api/atm/cuentas/101/saldo
{"type":"about:blank","title":"Service Unavailable","status":503,"detail":"Cajero sin conexion con el banco, intenta mas tarde","instance":"/api/atm/c
uentas/101/saldo"}  503
~/duoc/backend3/repo/s9  main ±
```

Captura 10: 10-resilience4j-fallback



Captura 11: 11-circuitbreakers-open



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.